# SE-JEPA: Self-Evolving Latent World Models

Anonymous CVPR submission

## Abstract

*Latent world models enable action planning but struggle with distribution shifts after deployment. Test-time adaptation addresses these shifts within an episode, yet resets afterward, forcing agents to repeatedly relearn recurring changes. We find that carrying adapted weights forward fails: consecutive updates are nearly orthogonal (mean cosine 0.018), and inheriting them cuts success from 62% to 48%. We therefore retain the observed transitions, not the weight updates. Inspired by complementary learning systems, we propose SE-JEPA, which pairs fast within-episode adaptation with slow cross-episode consolidation. The original pretrained weights remain fixed, while a separate slow predictor for each self-discovered domain learns from replayed trajectories. Its own prediction errors guide what to store and when to trust a learned predictor. No shift labels or task boundaries are needed, and both gradient-based and sampling-based planners benefit. SE-JEPA achieves state-of-the-art results, improving held-out success over AdaJEPA by 15.4–17.1 percentage points on unseen PushObj shapes and 15.9–18.7 points on PushT visual shifts. Relative to pretrained DINO-WM, it reduces Chamfer distance by 48.6% on rope and 26.7% on granular, with continued gains over deployment. The source code will be made public.*

## 1. Introduction

Robots deployed in changing environments should learn from repeated experience. After discovering how an unfamiliar object responds to a push, an agent should be able to use that knowledge when asked to move it to a different target. Latent world models support this reuse by predicting action outcomes, which a planner can evaluate against the current goal [2, 18, 22, 42]. Yet unfamiliar shapes or visual conditions can make these predictions inaccurate. The resulting plans can fail when executed.

Test-time adaptation addresses this mismatch by learning from the outcomes of the agent's own actions. AdaJEPA [35], for example, updates the model after each executed action block and replans with the corrected predictions. This improves planning within an episode, but the adapted weights are reset before the next goal. When the same change returns, the agent must learn to handle it again. Experience from earlier goals could instead help the agent handle these recurring conditions.

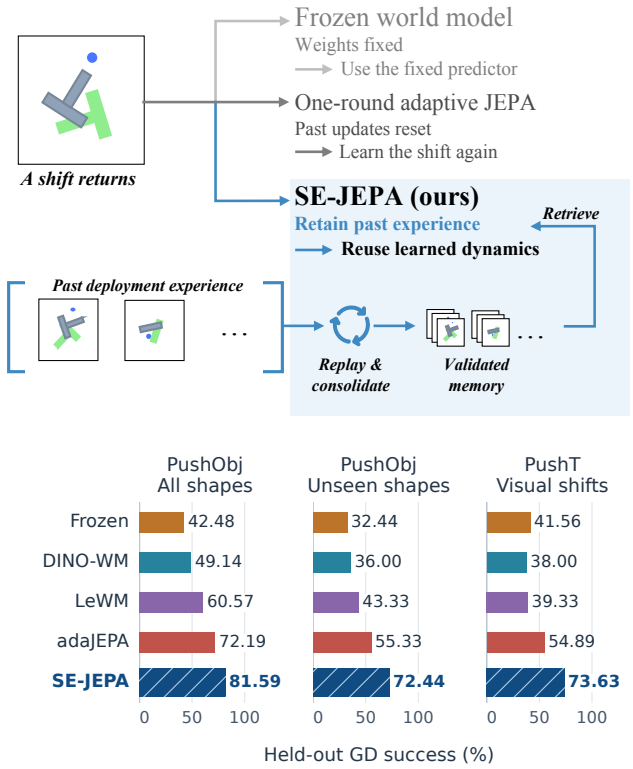



Figure 1. **Planning when a shift returns.** The schematic shows how SE-JEPA retrieves a predictor learned from earlier encounters. Bars show held-out GD success, ordered by all-shape PushObj success. One-round adaptive JEPA uses episode-local adaptation [35]. DINO-WM and LeWM use 50 targets per shape or visual condition. SE-JEPA reports the mean over three learning chains.

Keeping the adapted weights appears to be a simple solution, but our experiments expose a difficulty. Consecutive fast updates are nearly orthogonal, with mean cosine similarity 0.018. On a matched stream of 100 PushObj goals, carrying these updates forward reduces success from 62% to 48% (Sec. 3.1). An update fitted to one sequence of contacts need not help with the next. The observed transitions,

however, still record how the object responded to actions. Even an unsuccessful push can provide useful dynamics evidence for a later goal. These findings motivate learning across episodes from stored trajectories instead of accumulating their parameter updates.

We introduce SE-JEPA, which turns deployment trajectories into persistent predictive memory. Inspired by complementary learning systems [23], it combines fast adaptation within each episode with slower consolidation across episodes. The fast model adapts a working copy for the current encounter. The slow system replays stored transitions to train separate predictors for future goals, keeping the original pretrained weights fixed. SE-JEPA groups compatible trajectories into self-discovered *predictive domains*, each with its own replay buffer and slow predictor. It requires no labels identifying shifts or task boundaries.

Persistent memory must also be selective: a newly trained predictor may not yet be reliable enough to guide actions. SE-JEPA uses prediction errors to decide both what to retain and when to reuse it. Episodes that the pretrained model already predicts well need no direct memory write. A slow predictor becomes eligible for retrieval only after it consistently improves on the pretrained reference on trajectories scored before replay training. At the start of a later episode, recent observations determine which eligible predictor should guide planning (Fig. 1). Thus the agent can reuse past experience while checking that it remains relevant to the current encounter.

Experiments show benefits in both held-out planning and continued deployment. On unseen PushObj shapes and PushT visual shifts, SE-JEPA improves held-out success over AdaJEPA by 15–19 percentage points with gradient descent (GD) and the cross-entropy method (CEM). Under GD, it solves more goals halfway through the interaction budget than AdaJEPA solves with the full budget. A single memory also learns across all 14 shape and visual domains, raising online success from 65.3% to 75.5% over five rounds (Sec. 4.5). With a DINO-WM backbone, replay reduces rope and granular goal error without within-episode adaptation. A rope predictor learned in one pass reduces Chamfer distance by 47% on new instances, demonstrating generalization to previously unobserved trajectories.

## 2. Related Work

**World models for control.** World models support predictive planning [6, 9, 37], policy learning in learned simulators [8, 10–12, 16, 24], tree search [29, 38], and latent control with value learning [13, 14]. We focus on goal-image planning, where executed actions provide transition errors for improving the deployed predictor.

**Predictive representations and JEPA.** Predictive learning [18, 30, 32] complements view matching and embedding regularization [3, 7, 40]. Image and video JEPA models [1, 2, 4] build on learned visual features [5, 25, 26], with recent extensions to action-conditioned planning. Recent world models study feature learning, latent geometry, and separating object dynamics from visual backgrounds [22, 36, 39, 42]. SE-JEPA uses deployment trajectories to improve planning across recurring shifts.

**Test-time adaptation.** Test-time training and entropy minimization update deployed models from unlabeled observations [31, 33], including under changing distributions [34]. In control, each executed action supplies a target for the dynamics predictor. AdaJEPA [35] uses this signal within an episode and resets the weights at its end. JEPA-TTT [41] studies a dynamics shift that persists across episodes. It trains one predictor by replaying overlapping windows from the accumulated interactions. SE-JEPA addresses recurring shifts. It keeps the pretrained weights fixed, groups compatible trajectories, and retrieves separate slow predictors whose errors on new trajectories are lower than those of the pretrained reference. These comparisons precede replay training on each trajectory.

**Continual learning and complementary memory.** Complementary learning systems pair rapid acquisition with slow consolidation [23]. Continual learners replay earlier experience [27], constrain updates to protect past tasks [17, 21], preserve earlier outputs through distillation [19], or retain separate task modules [28]. In planning, predictor selection also affects subsequent actions and observations. SE-JEPA selects among validated predictors and the pretrained model using the episode's first observed transitions.

## 3. SE-JEPA

### 3.1. Motivation

Consider two episodes in which the agent pushes the same unfamiliar object toward different goals. The first may reveal how an off-center push rotates the object. That observation can help the agent choose a push in the second episode, even when it approaches the object from a different side. An episodic learner corrects its predictions along the first trajectory [35]. Reusing the adapted weights assumes that the correction also benefits the next trajectory.

We compared these choices on the same stream of 100 PushObj goals. Inheriting the previous episode's fast weights recovered 11 goals that a reset had missed, but lost 25 that the reset had solved (Table 2). In a separate seven-round stream, the mean cosine between consecutive fast-update fingerprints is 0.018 for both GD and CEM, over 2,449 pairs per planner. Absolute cosine is below 0.1 for 96.3% and 96.8% of the pairs, respectively (Appendix D). A stored trajectory preserves the observed action outcomes

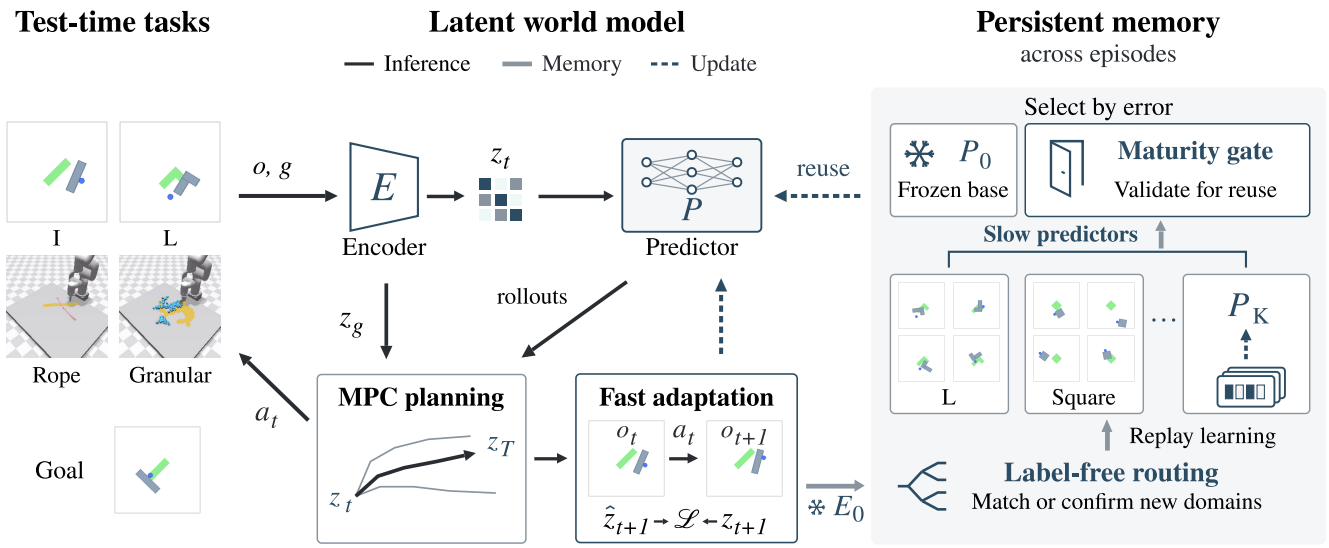



Figure 2. **SE-JEPA architecture.** The fast loop plans in latent space and adapts the predictor from observed transitions. At the end of an episode, the slow system re-encodes the trajectory with the frozen encoder, assigns unfamiliar experience to a self-discovered domain, and trains that domain's slow predictor by replay. Predictors that pass the maturity gate compete with the frozen base at the start of the next episode. Fast parameters reset at every episode; slow predictors and replay persist. Shape names illustrate memory contents and are not supervision. Test-time examples include rigid objects, rope, and granular material.

directly. A push that moves the object away from its goal still reveals how that contact changes its pose. If a later goal calls for that rotation, the planner benefits from having a predictor that can anticipate it. The learning signal is available as soon as the next observation arrives, regardless of whether the episode eventually succeeds.

SE-JEPA keeps a fast model for the current episode and trains a slow predictor on trajectories from multiple encounters (Fig. 2). The agent receives no labels identifying recurring shapes or visual conditions. It groups trajectories using their update directions and the ability of a shared predictor to model their transitions. When a new goal arrives, the first few actions provide evidence for which predictor to use.

### 3.2. Fast adaptation and replay consolidation

Each episode begins with working copies of the pretrained encoder $E_0$ and predictor $P_0$; their original weights remain fixed. The agent encodes the current observation and goal, rolls out candidate action sequences, and selects the plan with the lowest latent goal cost [22, 42]. It executes the first action block. Following AdaJEPA [35], it then takes one Adam step on the prediction loss over recent transitions to update the predictor's last block and the encoder head before replanning. These fast changes last for the episode. During the first few replans, the agent can also retrieve a slow predictor as a base for adaptation (Sec. 3.4).

Across episodes, SE-JEPA maintains a replay buffer $\mathcal{R}_k$ and a slow predictor $P_{\phi_k}$ for each group $k$ of compatible trajectories, which we call a *predictive domain*. After an episode, the frozen encoder $E_0$ re-encodes the executed trajectory. This gives replay a consistent latent space despite changes to the fast encoder within episodes. Once a trajectory is assigned to a domain, the agent adds it to $\mathcal{R}_k$. The slow predictor trains on windows sampled from that buffer by minimizing

$$\mathcal{L}_k(\phi_k) = \mathbb{E}_{(x,z^+)\sim\mathcal{R}_k} \left\| P_{\phi_k}(x) - z^+ \right\|_2^2. \tag{1}$$

Here $x$ contains a short history of observation embeddings and executed actions, and $z^+$ is the embedding of the next observation. Each slow predictor is a separate copy of $P_0$ trained by replay. A replay update samples episodes from the buffer and draws a short window from each. The same update can therefore include a push that translated an object and an earlier push that turned it. Both windows describe observed motion in the coordinates of $E_0$, even if fast adaptation changed the encoder differently in the two episodes.

### 3.3. Discovering predictive domains

The frozen reference first tests whether an episode contributes new experience. Let $\ell_{e,t}^{(0)}$ be its next-frame prediction error, and $\mathcal{I}_e$ index up to the first three observed transitions. Their geometric mean is

$$F_e = \exp\left( \frac{1}{|\mathcal{I}_e|} \sum_{t\in\mathcal{I}_e} \log \ell_{e,t}^{(0)} \right). \tag{2}$$

If $F_e < \tau_f$, the calibrated familiarity threshold, the agent keeps $P_0$ as its adaptation base and skips the episode's direct

memory write. Replay thus receives episodes the reference cannot yet predict accurately.

For an unfamiliar episode, $\Delta_e$ concatenates the fast predictor and encoder-head changes. Its normalized direction and similarity to domain $k$ are

$$u_e = \frac{\Delta_e}{\|\Delta_e\|_2}, \qquad s(e,k) = u_e^\top c_k, \tag{3}$$

where $c_k$ is the domain's unit-length prototype. The retrieved domain is the first write candidate; otherwise, candidates are ranked by $s(e,k)$. The trajectory is assigned to a candidate domain if its predictor passes the error check in Sec. 3.4. Unassigned trajectories remain in a pending pool. When a new direction aligns with those of two waiting episodes, the three trajectories establish a new domain.

Two contacts with the same object can induce different fast updates. Delayed domain creation retains unusual interactions until similar evidence arrives. The new predictor can then train on trajectories from several encounters. As more evidence arrives, two domains merge if each predictor transfers to the other's recent episodes and both outperform the reference. This bidirectional check requires the merged domain to model both groups of trajectories. A mature domain can also absorb a smaller group whose trajectories it already predicts well. Appendix A gives the thresholds and the rules for maintaining each prototype.

### 3.4. Validated retrieval for planning

A slow predictor becomes eligible for planning by predicting new trajectories accurately before training on them. Before adding trajectory $\tau_e$ to a buffer, the agent rolls out the candidate predictor and the pretrained reference under the actions actually taken. Their open-loop errors in the frozen latent space give the ratio

$$r_{e,k} = \frac{\mathcal{L}_{\text{roll}}(\phi_k; \tau_e)}{\mathcal{L}_{\text{roll}}(\theta_0; \tau_e)}. \tag{4}$$

Here $\theta_0$ denotes the weights of $P_0$. Scoring both predictors on the same actions controls for differences in trajectory difficulty. For example, a difficult turn may cause large errors in both models, yet still provide evidence for the slow predictor if its error is lower. The trajectory is rejected if the ratio exceeds the write threshold. Accepted episodes build a validation history of the predictor's performance on data it had not yet trained on. A domain becomes eligible for retrieval once this record shows a sustained improvement over the reference (Appendix A).

During the first three replans, the agent compares eligible predictors and $P_0$ on the transitions observed so far, $\mathcal{B}_{e,t}$. It selects the predictor with the lowest recent error,

$$k^\star = \underset{k \in \{0\} \cup \mathcal{K}_{\text{eligible}}}{\arg\min} \; \mathcal{L}_{\text{pred}}(\phi_k; \mathcal{B}_{e,t}), \qquad \phi_0 = \theta_0. \tag{5}$$

The selected weights become the base of the fast predictor, preserving the corrections already made in the current episode. For an episode the reference recognizes as familiar, the base stays at $P_0$. Past trajectories therefore determine retrieval eligibility, while current observations determine which predictor guides planning.

## 4. Experiments

### 4.1. Evaluation setup

**Tasks and backbones.** The planner receives an image goal but no shift labels or task-change signals. We study shape and appearance shifts with task-trained ResNet world models [15]. *PushObj* pretraining covers T, L, Z, and plus shapes; deployment also introduces I, square, and small tee. *PushT* keeps the T shape and changes appearance through blur, noise, darkness, or recoloring of the agent, object, or goal. Deformable tasks, *rope* and *granular*, use DINO-WM with pretrained DINOv2 features [20, 26, 42].

**Deployment and evaluation.** The separate-task PushObj and PushT comparisons use three learned states per planner and five rounds, resampling 50 targets per condition per round. Table 1 reports fifth-round online PushObj results for SE-JEPA. We also evaluate each learned memory on separate test-seed streams with memory writes disabled and retrieval and fast adaptation active. We report success and mean replans, counting the full budget for failures. Rope and granular isolate slow consolidation: one predictor per task revisits 50 fixed instances over four passes, with fast adaptation disabled. We measure final Chamfer distance (CD) to the goal and separately test rope on new instances.

**Baselines and planners.** We compare with Frozen, official-checkpoint AdaJEPA [35], and the predictive and representation baselines in Table 1. Rigid-object planners share a 150-action budget. Frozen, AdaJEPA, and SE-JEPA use paired held-out targets; Appendix B and Appendix C specify checkpoints and evaluation groups. Published deformable baseline scores come from Zhou et al. [42] and use ten instances per task; Appendix C specifies the evaluation groups and checkpoint coverage.

### 4.2. Planning with memory on held-out targets

**Generalization across shapes and appearances.** On shapes absent from pretraining, held-out CEM success is about 15 percentage points higher with SE-JEPA than with AdaJEPA (Table 5). Both planners gain most on I, followed by square and small tee (Fig. 7; Table 6). Stored transitions support planning toward new goals.

On PushT, CEM success under the red-object shift rises by over thirty percentage points relative to both Frozen and AdaJEPA, which tie on this condition (Table 7). Red-goal

Table 1. **Goal-conditioned planning with CEM.** PushObj covers seven shapes, three unseen in pretraining. The SE-JEPA PushObj cells report fifth-round online performance over three chains; baseline cells use separate benchmark evaluations. PushT covers six visual shifts, including three recolorings. Success rates are percentages. Steps use five-action units, with failures assigned 30 (Appendix B). Appendix C gives checkpoint coverage and sources for published Chamfer distance (CD) results.

| | PushObj | | | PushT visual shifts | | | Rope | Granular |
|---|---|---|---|---|---|---|---|---|
| Method | All (7) ↑ | Unseen ↑ | Steps ↓ | Shift (6) ↑ | Color (3) ↑ | Steps ↓ | CD ↓ | CD ↓ |
| DreamerV3 [12] | 7.1 | 5.3 | 28.36 | 5.7 | 0.0 | 28.91 | 2.49 | 1.05 |
| IRIS [24] | 2.9 | 2.7 | 29.37 | 6.0 | 6.7 | 28.85 | 1.11 | 0.37 |
| R3M [25] | 18.9 | 18.0 | 25.49 | 21.7 | 27.3 | 25.00 | 1.13 | 0.95 |
| DINO CLS [26] | 14.9 | 10.0 | 26.77 | 34.7 | 35.3 | 22.38 | 0.84 | 0.79 |
| DINO-WM [42] | 62.0 | 46.0 | 16.15 | 33.0 | 37.3 | 22.05 | 0.491 | 0.301 |
| LeWM [22] | 64.0 | 50.0 | 16.28 | 40.3 | 16.0 | 21.45 | 0.786 | 0.322 |
| DDP-WM [39] | 57.7 | 43.3 | 17.59 | 52.7 | 57.3 | 18.39 | 0.677 | 0.259 |
| Frozen (pretrained) | 43.8 | 36.2 | 21.08 | 48.3 | 33.6 | 20.10 | 0.491 | 0.301 |
| AdaJEPA [35] | 70.4 | 54.4 | 16.02 | 60.2 | 42.2 | 18.46 | 0.629 | 0.304 |
| SE-JEPA | **80.4** | **73.1** | **12.99** | **76.1** | **65.2** | **14.46** | **0.252** | **0.220** |




Figure 3. **Persistent memory resolves planning failures.** Selected GD episodes: unseen I (top) and mixed-stream red object (bottom), with matched starts and goals. Labels count MPC replans; **SE-JEPA** succeeds in six. All runs allow 30 replans of five actions each. Appendix E details the mixed-stream and replay protocols.

recoloring also shows a large gain. Replay reuses experience with each visual shift across goals.

**Planning efficiency.** Halfway through the GD replan budget, SE-JEPA solves more goals than AdaJEPA reaches with the full budget, on both the seven shapes and seven visual conditions (Fig. 5). Curves include all goals, including failures. CEM gives the same ordering in mean replans for both shapes and visual shifts (Appendix B).

**Consolidation on a second backbone.** With the same 15-push budget, the consolidated DINO-WM predictor lowers final CD on both rope and granular (Table 1). Fast adaptation is disabled, so these gains come from consolidation of earlier trajectories. Replay therefore benefits both task-trained ResNet and pretrained DINOv2 representations.

### 4.3. Qualitative results

With matched starts and goals, SE-JEPA solves unseen-I and mixed-stream red-object episodes missed by Frozen, DINO-WM, and AdaJEPA (Fig. 3). The slow predictor alone also moves rope and particles closer to their target configurations (Fig. 4). Appendix E gives full trajectories, prediction losses, and task outcomes.

### 4.4. Learning over deployment

With targets resampled each round, both GD and CEM improve online success on unseen shapes and recolored scenes from the first to final round (Fig. 6, top). Table 1 reports fifth-round online PushObj results; Appendix B gives paired evaluations after learning.

Revisiting rope and granular reduces CD over successive passes (Fig. 6, bottom). After one rope pass, we froze the

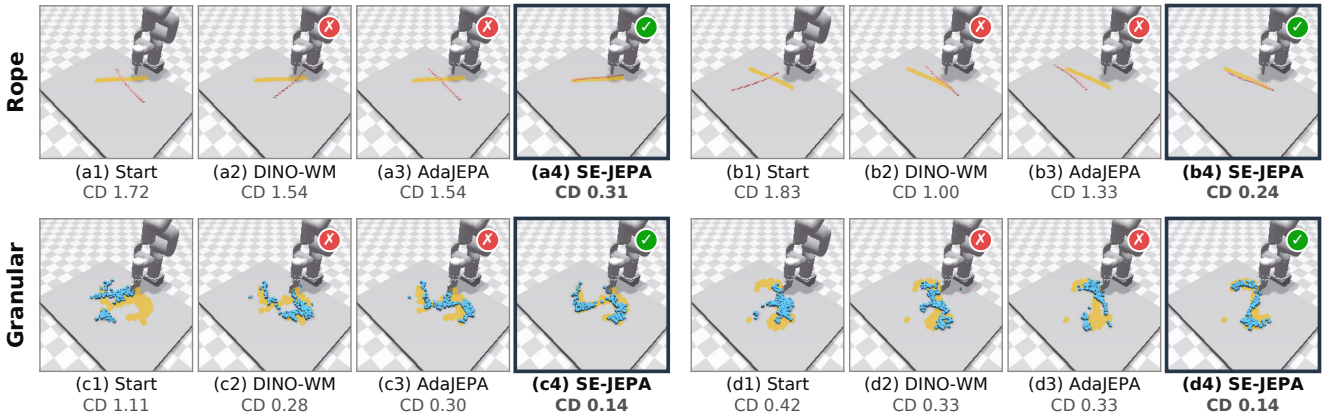



Figure 4. **Consolidated predictions support deformable manipulation.** Real FleX simulator trajectories for rope (top) and granular material (bottom), with two instances per task. Each group shows the start and the three methods' final states after 15 pushes; gold denotes the goal. These third-pass examples illustrate successful slow-only consolidation and were selected where both baselines fail. Frozen DINO-WM frames replay its executed actions. The aggregate fourth-pass results are reported in Table 1.

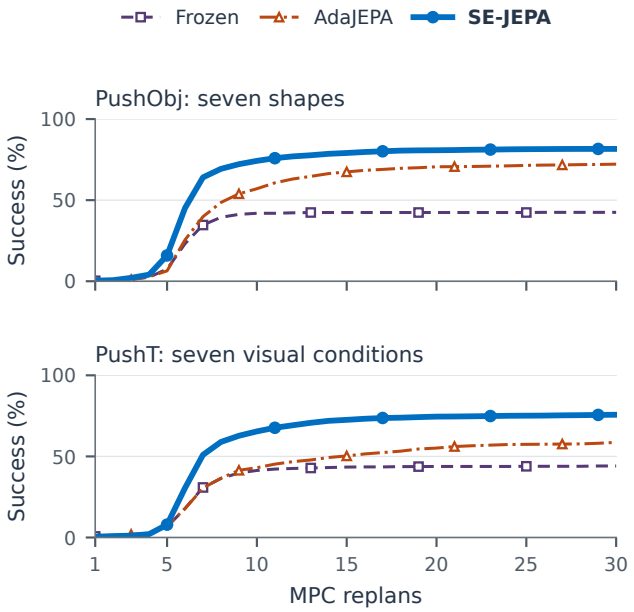



Figure 5. **Higher success within the same interaction budget.** Cumulative held-out success under GD. Curves use all seven shapes (top) and all seven visual conditions (bottom).

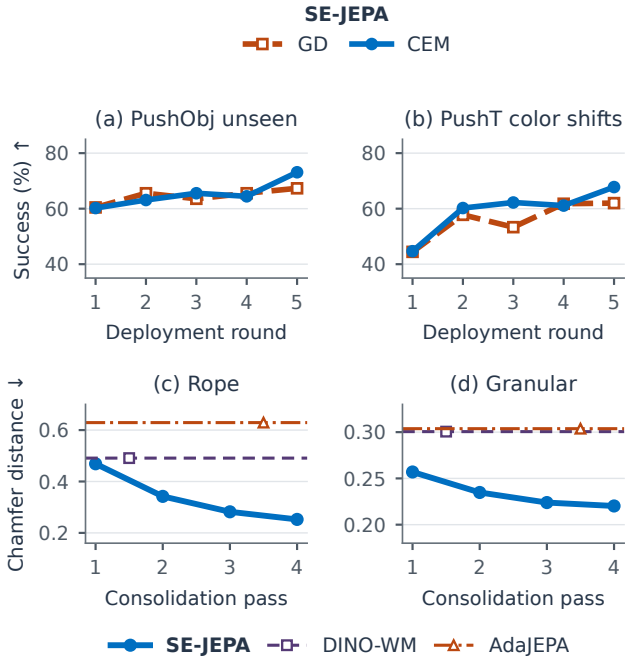



Figure 6. **Deployment experience improves two backbone families.** Top: ResNet-based models, with five rounds of online success on unseen PushObj shapes and three PushT color shifts; curves average three learning chains with targets resampled each round. Bottom: DINO-WM, with four consolidation passes over 50 fixed rope or granular instances; curves show mean CD. The horizontal lines are frozen DINO-WM and AdaJEPA.

predictor and tested 91 new instances. Mean CD fell from pretrained DINO-WM's 0.664 to 0.349, a 47% reduction; the paired bootstrap 95% interval for the mean reduction is [0.162, 0.470] (Appendix B). Both predictors use the same new starts and goals, with weights fixed.

**Selective writing and retrieval.** Across seven PushObj rounds, about one quarter of episodes contribute to replay, and a similar fraction use a slow predictor at their final replan. On the four pretraining shapes, both planners also outperform AdaJEPA while retaining the original model as an adaptation base (Appendix D).

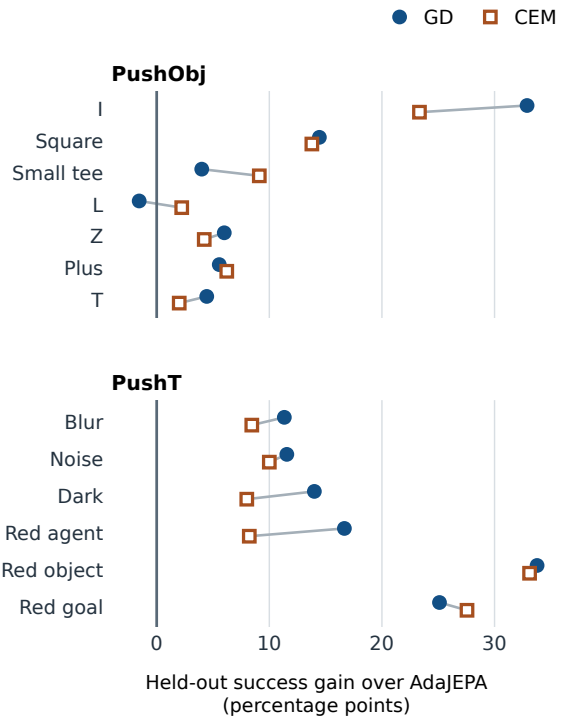



Figure 7. **Planning gains by shape and visual condition.** PushObj (top) and PushT (bottom). Each point gives full SE-JEPA minus AdaJEPA held-out success, in percentage points. Positive values favor SE-JEPA. I, square and small tee are absent from the backbone's pretraining trajectories.

### 4.5. One memory across task and shift families

We combine seven PushObj shapes and seven PushT visual conditions in one 14-domain stream. Each of three chains per planner starts with an empty memory and carries it through all domains for five rounds, with 50 episodes per domain per round. Targets repeat, while adaptation and memory updates remain active.

Averaging GD and CEM equally, online success rises from 65.3% to 75.5% (+10.2 points; Fig. 8). Both families improve: shapes from 73.4% to 82.0% and visual conditions from 57.2% to 68.9%. One evolving memory thus supports recurring geometric and appearance shifts across task families without resets. The AdaJEPA and Frozen references reach 65.2% and 38.2%; they use different initial checkpoints and do not retain adaptation across episodes. Appendix D provides the protocols and detailed results for each shape and visual condition.

### 4.6. Ablation study

**Accumulating fast updates across episodes.** We test whether retaining a fixed fraction of each fast update benefits subsequent goals. We compare $s_{e+1} = s_e + w\Delta_e$, where $s_e$ is the starting offset from the pretrained weights and $\Delta_e$ is the episode's fast update. Every setting receives the same 100 goals in five ordered blocks (I, square, T, I, square). Resetting between episodes reaches 62% of goals. Full inheritance reaches 48%, losing 25 reset successes and recovering 11 (Table 2, exact McNemar $p = 0.029$). The smallest tested coefficient reaches 64%, while larger coefficients progressively reduce success. Thus the degradation depends on how strongly past updates are accumulated.

Table 2. **Fast-update accumulation on 100 matched goals.** The coefficient $w$ scales the update retained between episodes. Wins and losses are relative to episodic reset.

| Accumulation $w$ | Success (%) | Wins / losses |
|---|---|---|
| 0 (episodic reset) | 62.0 | — |
| 0.003 | 64.0 | 7 / 5 |
| 0.01 | 59.0 | 5 / 8 |
| 0.03 | 54.0 | 4 / 12 |
| 1 (naive inheritance) | 48.0 | 11 / 25 |

**Replay rather than parameter averaging.** Table 3 tests corresponding five-round learning streams from the same task-specific checkpoint, with 1,050 held-out evaluations per state. PushObj uses one chain; PushT repeats the components in two. The episodic-reset baseline removes persistent memory while keeping the same initial weights; the main benchmark uses the official AdaJEPA checkpoint.

Shared replay exceeds fast-parameter EMA by 5.52 points on PushObj and by 10.10 and 8.86 points on the two PushT chains. On PushObj, replay solves 44 additional unseen-I goals out of 150, while the four pretraining shapes differ by only one solved goal. The full model also exceeds episodic reset in every comparison, supporting retention of transitions rather than episode-specific weight corrections.

**Inheritance across repeated learning chains.** Across three chains, inheritance averages 65.94% success versus 81.59% for SE-JEPA on PushObj, and 31.71% versus 75.68% on PushT. The full model leads in every chain. Mean replans, including failures, fall from 15.84 to 11.30 on shapes and from 24.18 to 13.21 on visual conditions (Appendix D). Each independently learned memory improves success and reduces interaction costs.

**Fixed slow learning rates.** Rates from $5 \times 10^{-5}$ to $3 \times 10^{-4}$ give 79.62–82.38% on PushObj and 73.05–78.10% across the two PushT chains. A rate of $5 \times 10^{-4}$ reduces success in all three comparisons relative to the default $10^{-4}$. Doubling the default changes the three-chain visual mean by only $+0.19$ points. These results support a range of slow learning rates; increasing the update size does not consistently improve planning success.

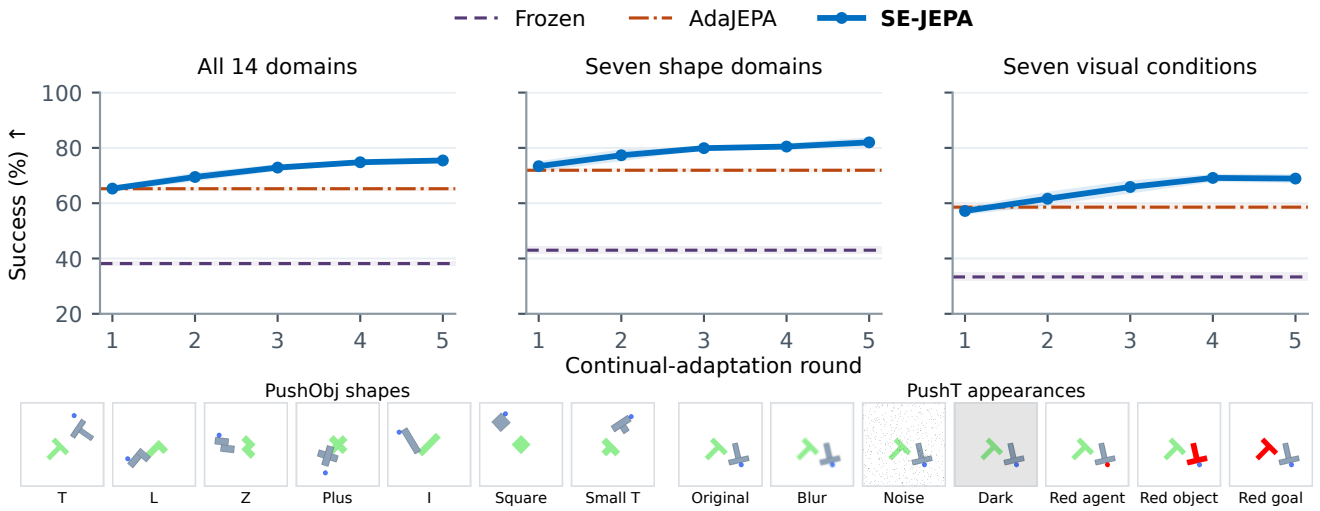



Figure 8. **One memory learns across 14 domains and two task families.** Bottom: recorded observations from each domain. Curves average GD and CEM equally; SE-JEPA retains one memory across five rounds with repeated targets. Frozen and AdaJEPA are archived evaluation references. Shading: sample SD across six runs. Initializations and adaptation exposure differ (Appendix D).

Table 3. **GD component and learning-rate ablations.** Success (%) on seven domains per task, 1,050 held-out evaluations per cell. C1/C2 denote learning chains. The PushT reset score is one shared evaluation, not an independent run per column. Appendix D gives replans and additional chains.

| | PushObj | PushT | |
|---|---|---|---|
| Variant | C1 | C1 | C2 |
| Full SE-JEPA | 81.43 | 74.38 | 76.57 |
| Episodic reset | 76.95 | 65.90 | 65.90 |
| Inherit fast weights | 64.67 | 29.14 | 40.19 |
| Shared predictor: EMA | 75.71 | 66.10 | 66.29 |
| Shared predictor: replay | 81.24 | 76.19 | 75.14 |
| No maturity screening | 80.86 | 76.95 | 78.10 |
| No familiar-read protection | 76.95 | 74.67 | 76.19 |
| No familiar-write protection | 79.43 | 71.62 | 73.81 |
| Neither familiarity protection | 83.14 | 72.76 | 73.14 |
| Full SE-JEPA: fixed slow learning rate | | | |
| $5 \times 10^{-5}$ | 79.62 | 75.24 | 73.05 |
| $10^{-4}$ (default) | 81.43 | 74.38 | 76.57 |
| $2 \times 10^{-4}$ | 82.38 | 74.86 | 76.86 |
| $3 \times 10^{-4}$ | 80.57 | 74.29 | 78.10 |
| $5 \times 10^{-4}$ | 78.38 | 71.62 | 74.38 |

**Maturity and familiarity gates.** Removing maturity screening lowers PushObj success from 81.43% to 80.86%, but raises PushT success from 74.38% to 76.95% and from 76.57% to 78.10%. A prediction-error gate can filter unreliable predictors but may also delay useful retrieval, leaving room to improve its calibration.

The familiarity checks keep the pretrained model as the adaptation base for familiar episodes and prevent these episodes from directly entering replay. Disabling either protection alone lowers success in all three comparisons. Disabling both raises PushObj success to 83.14%, but lowers the three-chain PushT mean from 75.68% to 72.32%, with every visual chain declining. The same gate can therefore affect shape and visual shifts differently.

## 5. Conclusion

SE-JEPA consolidates informative deployment trajectories into predictors validated for future planning. It improves held-out success with GD and CEM and learns across 14 shape and visual domains with one memory. Replay also reduces deformable goal error, and a rope predictor learned in one pass generalizes to new instances.

**Future work.** Prediction uncertainty and planning outcomes could help the maturity gate determine when a learned predictor is ready for use. Longer observation histories and exploratory actions could distinguish similar dynamics. Better action selection near the goal could turn prediction improvements into successful plans. Longer deployment studies should measure memory growth, retrieval latency, and replay cost alongside planning success. Comparing memory compression with retaining every predictor would help identify practical limits on storage and computation. Tests with rare but recurring conditions would also reveal whether compact memories preserve useful experience or discard it before it is needed again.

## LLM Usage Statement

Large language models (OpenAI Codex and Anthropic Claude) were used to draft and revise substantial portions of the manuscript text, and to assist with literature and source checks, LaTeX organization, figure generation, and result aggregation. They also assisted with experimental code, including offline world-model adapters and experiment orchestration. All quantitative results are taken from saved experimental outputs or explicitly cited published sources; language-model output is not used as experimental evidence. The authors take full responsibility for the claims, citations, code, and final verification of the manuscript.

## References

[1] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023. 2

[2] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, et al. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025. 1, 2

[3] Adrien Bardes, Jean Ponce, and Yann LeCun. VICReg: Variance-invariance-covariance regularization for self-supervised learning. In *International Conference on Learning Representations*, 2022. 2

[4] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video. *arXiv preprint arXiv:2404.08471*, 2024. 2

[5] Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. Emerging properties in self-supervised vision transformers. In *IEEE/CVF International Conference on Computer Vision*, 2021. 2

[6] Kurtland Chua, Roberto Calandra, Rowan McAllister, and Sergey Levine. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. In *Advances in Neural Information Processing Systems*, 2018. 2

[7] Jean-Bastien Grill, Florian Strub, Florent Altché, Corentin Tallec, Pierre H. Richemond, Elena Buchatskaya, Carl Doersch, Bernardo Avila Pires, Zhaohan Daniel Guo, Mohammad Gheshlaghi Azar, Bilal Piot, Koray Kavukcuoglu, Rémi Munos, and Michal Valko. Bootstrap your own latent: A new approach to self-supervised learning. In *Advances in Neural Information Processing Systems*, 2020. 2

[8] David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018. 2

[9] Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *International Conference on Machine Learning*, pages 2555–2565, 2019. 2

[10] Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations*, 2020. 2

[11] Danijar Hafner, Timothy Lillicrap, Mohammad Norouzi, and Jimmy Ba. Mastering atari with discrete world models. In *International Conference on Learning Representations*, 2021.

[12] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 2025. 2, 5

[13] Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations*, 2024. 2

[14] Nicklas A. Hansen, Hao Su, and Xiaolong Wang. Temporal difference learning for model predictive control. In *International Conference on Machine Learning*, pages 8387–8406, 2022. 2

[15] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016. 4

[16] Lukasz Kaiser, Mohammad Babaeizadeh, Piotr Milos, Blazej Osinski, Roy H. Campbell, Konrad Czechowski, Dumitru Erhan, Chelsea Finn, Piotr Kozakowski, Sergey Levine, Afroz Mohiuddin, Ryan Sepassi, George Tucker, and Henryk Michalewski. Model-based reinforcement learning for atari. In *International Conference on Learning Representations*, 2020. 2

[17] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017. 2

[18] Yann LeCun. A path towards autonomous machine intelligence. OpenReview position paper, 2022. 1, 2

[19] Zhizhong Li and Derek Hoiem. Learning without forgetting. In *European Conference on Computer Vision*, pages 614–629, 2016. 2

[20] Xingyu Lin, Yufei Wang, Jake Olkin, and David Held. SoftGym: Benchmarking deep reinforcement learning for deformable object manipulation. In *Proceedings of the 2020 Conference on Robot Learning*, pages 432–448, 2021. 4

[21] David Lopez-Paz and Marc'Aurelio Ranzato. Gradient episodic memory for continual learning. In *Advances in Neural Information Processing Systems*, 2017. 2

[22] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorldModel: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026. 1, 2, 3, 5

[23] James L. McClelland, Bruce L. McNaughton, and Randall C. O'Reilly. Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3):419–457, 1995. 2

[24] Vincent Micheli, Eloi Alonso, and François Fleuret. Transformers are sample-efficient world models. In *International Conference on Learning Representations*, 2023. 2, 5

[25] Suraj Nair, Aravind Rajeswaran, Vikash Kumar, Chelsea Finn, and Abhinav Gupta. R3M: A universal visual representation for robot manipulation. In *Proceedings of the 6th Conference on Robot Learning*, pages 892–909, 2023. 2, 5

[26] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mahmoud Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jégou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2024. 2, 4, 5

[27] David Rolnick, Arun Ahuja, Jonathan Schwarz, Timothy Lillicrap, and Gregory Wayne. Experience replay for continual learning. In *Advances in Neural Information Processing Systems*, 2019. 2

[28] Andrei A. Rusu, Neil C. Rabinowitz, Guillaume Desjardins, Hubert Soyer, James Kirkpatrick, Koray Kavukcuoglu, Razvan Pascanu, and Raia Hadsell. Progressive neural networks. *arXiv preprint arXiv:1606.04671*, 2016. 2

[29] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, and David Silver. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588:604–609, 2020. 2

[30] Max Schwarzer, Ankesh Anand, Rishab Goel, R. Devon Hjelm, Aaron Courville, and Philip Bachman. Data-efficient reinforcement learning with self-predictive representations. In *International Conference on Learning Representations*, 2021. 2

[31] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In *International Conference on Machine Learning*, 2020. 2

[32] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018. 2

[33] Dequan Wang, Evan Shelhamer, Shaoteng Liu, Bruno Olshausen, and Trevor Darrell. Tent: Fully test-time adaptation by entropy minimization. In *International Conference on Learning Representations*, 2021. 2

[34] Qin Wang, Olga Fink, Luc Van Gool, and Dengxin Dai. Continual test-time domain adaptation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022. 2

[35] Ying Wang, Oumayma Bounou, Yann LeCun, and Mengye Ren. AdaJEPA: An adaptive latent world model. *arXiv preprint arXiv:2606.32026*, 2026. 1, 2, 3, 4, 5, 11

[36] Ying Wang, Oumayma Bounou, Gaoyue Zhou, Randall Balestriero, Tim G. J. Rudner, Yann LeCun, and Mengye Ren. Temporal straightening for latent planning. In *International Conference on Machine Learning*, 2026. 2

[37] Manuel Watter, Jost Tobias Springenberg, Joschka Boedecker, and Martin Riedmiller. Embed to control: A locally linear latent dynamics model for control from raw images. In *Advances in Neural Information Processing Systems*, 2015. 2

[38] Weirui Ye, Shaohuai Liu, Thanard Kurutach, Pieter Abbeel, and Yang Gao. Mastering atari games with limited data. In *Advances in Neural Information Processing Systems*, 2021. 2

[39] Shicheng Yin, Kaixuan Yin, Weixing Chen, Yang Liu, Guanbin Li, and Liang Lin. DDP-WM: Disentangled dynamics prediction for efficient world models. *arXiv preprint arXiv:2602.01780*, 2026. 2, 5

[40] Jure Zbontar, Li Jing, Ishan Misra, Yann LeCun, and Stephane Deny. Barlow twins: Self-supervised learning via redundancy reduction. In *International Conference on Machine Learning*, pages 12310–12320, 2021. 2

[41] Zheyuan Zhang, Suyu Ye, Nakul Agarwal, Hossein Nourkhiz Mahjoub, Ehsan Moradi Pari, Daniel Khashabi, Tianmin Shu, and Vaishnav Tadiparthi. JEPA-TTT: Persistent test-time training of latent world models for planning under dynamics shifts. *arXiv preprint arXiv:2610.00722*, 2026. 2

[42] Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. In *International Conference on Machine Learning*, pages 79115–79135, 2025. 1, 2, 3, 4, 5, 13

# Supplementary Material

## A. Implementation and Memory Management

**Compute environment.** Model training and evaluation used a server with eight NVIDIA A100 GPUs.

### A.1. Fast and slow learning

The label-free rigid-object configuration builds on the episodic adaptation loop of AdaJEPA [35]. At every replan, recent observed transitions provide a self-supervised next-state prediction loss. One Adam step adapts the predictor's last block and the encoder's designated projection/head parameters in working copies, leaving the original pretrained weights fixed. The episode-local predictor and encoder changes are reset at the next episode. A retrieved slow predictor supplies the predictor initialization; it does not replace the frozen reference encoder.

Persistent replay is re-encoded by the reference encoder. Each slow learner is a full predictor copy, initialized from the pretrained predictor rather than from the final fast parameters of the episode that created the domain. For each slow optimization step, up to eight stored episodes are sampled with replacement; each contributes a random window of up to six latent frames. The prediction objective averages one-step observation-feature errors over the available history windows. Actions condition the prediction; the loss is computed only on observation features. Standard writes use 15 Adam steps. Founding a learner after $n$ stored episodes uses $15n$ steps to process the accumulated evidence. Merging likewise adds replay updates for absorbed experience.

Replay stores compact latent trajectories rather than images. The pending pool is bounded, and merging and absorption remove redundant predictor copies as compatible domains are recognized.

### A.2. Order of decisions

Algorithm 1 summarizes a learning episode. Familiarity uses the geometric mean of newest-frame prediction errors on up to the first three observed transitions. The threshold is calibrated once on the shape backbone and transfers unchanged to the visual experiments.

The selected predictor's error is evaluated before it learns from the current episode, so maturity reflects transfer to data the predictor has not fitted. In read-only evaluation, the episode-end write, validation-history update, slow training, merging, and absorption steps are all disabled.

### A.3. Fingerprints and delayed domain founding

A fingerprint concatenates the adapted predictor and encoder-head changes and is normalized to unit length. Domain prototypes accumulate normalized fingerprints from episodes planned from the reference; episodes starting from a slow predictor do not redefine that reference-based prototype. If an episode retrieved a slow predictor, its trajectory is first considered for that predictor's domain, subject to the error-ratio check before replay. Otherwise, it tries compatible domain directions in descending cosine order.

An unassigned episode enters a pending pool. It confirms a new domain when at least two pending fingerprints align with its own above the new-domain threshold. These members need not be pairwise aligned with one another; the incoming fingerprint is the common reference for this confirmation. The members and the new episode are transferred together to the new replay. No ground-truth shape or appearance category participates in this procedure.

Table 4. Label-free configuration used for PushObj and PushT. The seven-round stream in Appendix D uses a slow learning rate of $2 \times 10^{-4}$ for all replay updates.

| Setting | Value |
|---|---|
| Fast predictor learning rate | $5 \times 10^{-4}$ |
| Fast encoder-head learning rate | $10^{-5}$ |
| Fast steps per replan | 1 |
| Recent adaptation buffer | 5 transitions |
| Slow learning rate | $10^{-4}$ |
| Slow steps per standard write | 15 |
| Slow batch size (episodes) | 8 |
| Slow sampled window (frames) | up to 6 |
| Familiarity threshold $\tau_f$ | 0.0146 |
| Existing-domain cosine threshold | 0.10 |
| New-domain cosine threshold | 0.05 |
| Episodes to confirm a domain | 3 |
| Pending-pool capacity | 40 |
| Minimum episodes to train a learner | 3 |
| Retrieval window | first 3 replans |
| Maturity history length $m$ | 3 |
| Maturity error-ratio threshold $\tau_m$ | 0.9 |
| Write rejection ratio | 1.2 |
| Merge evidence per direction | 2 episodes |
| Merge relative tolerance | 0.5 |
| Fragment absorption interval | 25 episodes |
| Absorption error-ratio threshold | 0.8 |
| Fragment-size limit | $< 6$ episodes |

### A.4. Merging and fragment absorption

For a new unfamiliar episode, every existing trained learner is evaluated before training. The resulting ratios provide cross-domain evidence. Let $G_{A\to B}$ be the geometric mean error ratio of learner $B$ on recent episodes assigned to $A$, and $G_{A\to A}$ the corresponding ratio for learner $A$. Domains merge only if both directions satisfy

$$G_{A\to B} \le \min\{1, (1+\epsilon_m)G_{A\to A}\}, \tag{6}$$

**Algorithm 1** One deployment-learning episode of SE-JEPA

**Require:** Frozen $(E_0, P_0)$, memory $\mathcal{M}$, initial observation $o$, goal $g$
Reset fast parameters to $(E_0, P_0)$
Initialize recent buffer $\mathcal{B}$ and episode trajectory $\tau$
**while** budget remains and task is not complete **do**
    Optimize an action sequence with the current fast model
    Execute its first block; observe the next state
    Append real transitions to $\mathcal{B}$ and $\tau$
    **if** within the early retrieval window **then**
        Form candidates: $P_0$ and mature slow predictors
        Restrict to $P_0$ if the observed episode is familiar
        Select the smallest observed prediction error
        Load the selected predictor, retaining the fast offset
    **end if**
    Adapt the fast model on recent real transitions
**end while**
Encode $\tau$ with frozen $E_0$
Measure pre-update rollout errors of existing predictors
**if** episode is unfamiliar **then**
    Propose a domain using retrieval or an update fingerprint
    Reject an incompatible write, or retain pending evidence
    **if** a domain is assigned or newly confirmed **then**
        Record validation before replay training when a learner exists
        Append latent experience; update its slow predictor
        Check bidirectional merges and periodic absorption
    **end if**
**end if**
Discard episode-local fast parameters

with the symmetric condition for $B \to A$. Each direction requires two jointly evaluated episodes in the primary configuration. The larger domain retains its learner and incorporates the union of stored experience. Identifiers of merged domains remain retired for the rest of the stream.

Every 25 episodes, small immature domains can be absorbed into a mature predictor that explains their experience with an error ratio at most 0.8. The primary configuration limits absorption to fragments with fewer than six episodes and fewer than three maturity measurements. Pending-pool absorption and the additional absolute-error gate are disabled in the shape and visual experiments.

# B. Evaluation Protocols

## B.1. Shape shifts

The shape learning stream has five deployment rounds over seven shapes, with 50 episodes per shape in every round. The post-learning benchmark uses target seeds 2000, 2100, and 2200, each with 350 episodes. I, square, and small tee are unseen in pretraining; T, L, Z, and plus are seen.

For the paired planner comparison in Table 5, each planner has three learned memory states. Every state is evaluated on all three test seeds. For each seed we average over episodes and states, then compute the standard deviation across seed-level means. The CEM states are S7t3, S7t3b, and S7t3s700. Figure 1 uses the paired GD results for Frozen, AdaJEPA, and SE-JEPA. The local external PushObj baselines use seed 2000. AdaJEPA uses its official released PushObj checkpoint for both planners.

The SE-JEPA PushObj entries in Table 1 instead summarize the fifth online round of these three CEM chains, with learning and memory writing active. Their success rates are 82.29%, 80.00%, and 78.86%, averaging 80.38%. Across 1,050 episodes, they reach 844 goals overall and 329 of the 450 goals on unseen shapes. Their initial/goal sets differ from the official baseline evaluation sets. The shape learning and test-seed streams are not strictly disjoint: an exact-state check found three shared initial/goal pairs between the selected fifth-round streams and the test set.

## B.2. Visual shifts

The paired visual evaluation contains the original T condition, blur, salt-and-pepper noise, darkness, red agent, red object, and red goal. Three five-round learning chains resample targets every round, with 350 episodes per round. Held-out testing uses three target seeds and 50 episodes per condition. Direct checks of source-episode identifiers confirm that held-out sources never occur in the learning streams and that the three test seeds use disjoint sources.

The color-learning curve in Fig. 6 aggregates red-agent, red-object, and red-goal episodes. For each round and learning chain, it sums successes over the three conditions and divides by 150 episodes, and plots the mean of the three chain-level rates. The archived statistics include their sample standard deviation. The three conditions share the same 50 source targets within a chain and round.

The external visual baselines use the established VIS50 collection with planning seed 100. The external comparisons use its six shifted conditions, with 50 paired initial/goal states per condition.

For the external PushT GD bars in Fig. 1, DINO-WM and LeWM share 50 targets per condition at seed 100. Shift averages the six altered conditions, excluding the original T. Both use 100 GD steps and a 150-physical-step cap. DINO-WM executes 25 physical steps per replan

(at most six replans); LeWM executes five (at most 30). Their checkpoints, training budgets and controllers differ, so these are method-level comparisons. AdaJEPA in the paired GD/CEM comparisons uses the official checkpoint released for visual-shift evaluation.

### B.3. Success and interaction counts

For the paired rigid-object comparisons, GD uses 100 optimization steps; CEM uses 10 iterations with 200 samples and 30 elites per iteration. Rigid-object success requires a joint Euclidean error below 20 simulator units across the pusher and object positions, and an object orientation error below $\pi/9$. Each executed model action contains five physical actions, and an episode has at most 30 MPC replans. For episode $i$ with success indicator $S_i$ and first successful replan $T_i$,

$$\overline{T} = \frac{1}{N}\sum_{i=1}^{N}\left[\, S_i T_i + (1 - S_i)\, T_{\max}\right], \qquad T_{\max} = 30. \quad (7)$$

Equivalently, $\overline{T} = T_{\max} - \sum_{t=1}^{T_{\max}-1} F(t)$, where $F(t)$ is the fraction of episodes solved by replan $t$; the cumulative curves in Fig. 5 visualize $F$.

DINO-WM PushT CEM executes 25 actions per replan; Table 1 converts its mean to five-action units, $4.41 \times 25/5 = 22.05$, with failures assigned 30 units.

### B.4. Deformable manipulation

Rope and granular each use one slow predictor, a DINO-WM backbone, and 50 instances revisited over four passes, with the same targets, order, and planning seeds in every pass. The slow-only variant plans from the slow predictor without fast within-episode updates. CEM uses 300 samples, 30 elites, and 10 iterations, with three MPC iterations of five pushes each. The unseen-rope evaluation freezes the predictor saved after the first pass and tests it on 91 instances absent from the learning stream, against the frozen reference on the same instances. Mean CD falls from 0.664 with the reference to 0.349 with the one-pass predictor. Across the 91 matched instances, the predictor improves CD on 56. A paired percentile bootstrap over instances (20,000 resamples, seed 20261107) gives a 95% interval of [0.162, 0.470] for the mean CD reduction. The observed mean reduction is 0.314. Two frozen granular episodes with unstable batched particle simulation were rerun individually with identical settings and seeds.

## C. Baseline Checkpoints and Comparisons

**Checkpoint provenance.** Frozen and AdaJEPA use the official task-specific checkpoints on PushObj and PushT. SE-JEPA uses task-trained ResNet world models; its PushObj backbone is trained for three epochs on 16,000 T, L, Z, and plus trajectories. The controlled ablations share this initial checkpoint, so their episodic-reset row isolates memory rather than reproducing the official-checkpoint benchmark. The additional DINO-WM shape model uses the same trajectory count and epoch budget. Its updated PushT CEM checkpoint uses 16,000 newly generated random T-shape trajectories and epoch three. DreamerV3, IRIS, DINO CLS, and R3M use T-only checkpoints. DreamerV3 and IRIS are frozen offline world-model adapters for continuous-action CEM planning, without reward or policy learning. Configurations and source records are archived with the paper.

**Evaluation groups.** Frozen, AdaJEPA, and SE-JEPA use the paired held-out targets in Appendix B. External visual baselines use VIS50: 50 initial/goal pairs per condition and seed 100. PushT Shift averages blur, noise, dark, red agent, red object, and red goal; Color averages the last three. The original condition enters the seven-condition curves. DreamerV3 and IRIS PushObj results use 50 targets per shape at seed 2000. Their Unseen column averages I, square, and small tee, matching the comparison group for other methods, although their T-only checkpoints have not seen any of the other six shapes. Methods within each group share targets and the total action budget.

**Published results and deformable references.** The CDs for DreamerV3, IRIS, R3M, and DINO CLS are reported by Zhou et al. [42] on ten instances per task. Our DINO-WM and Frozen deformable rows share the same fixed reference on 50 instances. Appendix B describes consolidation and the separate evaluation on unseen rope instances.

## D. Additional Results

### D.1. All object shapes

Table 6 reports every held-out shape. SE-JEPA gives the largest gains on the unseen shapes: with GD, I rises from 44.7% (AdaJEPA) to 77.6% and square from 46.7% to 61.1%. It also leads on the seen shapes in aggregate.

### D.2. All visual conditions

Table 7 and Fig. 7 break the paired visual results down by condition. SE-JEPA improves on all seven conditions with both planners. The largest gains, more than 25 points, appear under red-object and red-goal recoloring, where episodic adaptation has to learn the same appearance correction again for each new target.

### D.3. One memory across fourteen domains

This experiment places seven PushObj shapes and seven PushT visual conditions in one continual stream. The 14

Table 5. Both planners on the paired post-learning evaluation sets. Shape All and Unseen use the separate PushObj test-seed streams. Visual All averages seven conditions and Shift six; visual Steps averages all seven conditions.

| | | PushObj | | | PushT | | |
|---|---|---|---|---|---|---|---|
| Planner | Method | All ↑ | Unseen ↑ | Steps ↓ | All ↑ | Shift ↑ | Steps ↓ |
| GD | Frozen | 42.48 | 32.44 | 20.02 | 44.10 | 41.56 | 19.94 |
| GD | AdaJEPA | 72.19 | 55.33 | 14.52 | 58.67 | 54.89 | 17.96 |
| GD | SE-JEPA | 81.59 | 72.44 | 11.30 | 75.68 | 73.63 | 13.21 |
| CEM | Frozen | 43.81 | 36.22 | 21.08 | 50.86 | 48.33 | 19.56 |
| CEM | AdaJEPA | 70.38 | 54.44 | 16.02 | 62.95 | 60.22 | 17.87 |
| CEM | SE-JEPA | 79.08 | 69.85 | 13.53 | 77.05 | 76.11 | 14.15 |

Table 6. Held-out success (%) per PushObj shape. I, square, and small tee are unseen in pretraining. All averages the seven shapes.

| Planner | Method | I | L | Square | Z | Small tee | Plus | T | All |
|---|---|---|---|---|---|---|---|---|---|
| CEM | Frozen | 32.7 | 52.7 | 23.3 | 41.3 | 52.7 | 43.3 | 60.7 | 43.8 |
| CEM | AdaJEPA | 49.3 | 82.0 | 44.7 | 82.0 | 69.3 | 79.3 | 86.0 | 70.4 |
| CEM | SE-JEPA | 72.7 | 84.2 | 58.4 | 86.2 | 78.4 | 85.6 | 88.0 | 79.1 |
| GD | Frozen | 24.0 | 52.7 | 28.0 | 40.0 | 45.3 | 48.7 | 58.7 | 42.5 |
| GD | AdaJEPA | 44.7 | 90.0 | 46.7 | 82.0 | 74.7 | 82.0 | 85.3 | 72.2 |
| GD | SE-JEPA | 77.6 | 88.4 | 61.1 | 88.0 | 78.7 | 87.6 | 89.8 | 81.6 |

domains form two recurring families, rather than all shape–appearance combinations. Each round presents all seven visual conditions followed by all seven shapes, with the shape order alternating between rounds. One memory state carries through every domain and round. Three chains per planner start with an empty memory and run for five rounds, with 50 episodes per domain per round. The same initial/goal pairs recur throughout; the changing seeds control run randomness. Thus the curves measure online improvement on repeated targets, complementing the separate evaluation of held-out goal configurations.

**Improvement across both families.** Section 4.5 and Figure 8 report five-round success averaged equally over GD and CEM. The figure's horizontal baselines average the archived evaluations, not additional five-round trajectories; shading is the sample SD of six planner-by-chain or planner-by-seed success rates. Each online round totals 4,200 episodes across both planners. The thumbnail strip shows the recorded initial observation of episode zero in each domain, from GD chain two in round five. Selection uses a fixed episode index independent of success; the native images retain the actual appearance corruptions. Both families improve from the first to fifth round: GD rises from 56.8% to 69.1% on visual conditions and from 74.4% to 81.1% on shapes; CEM rises from 57.6% to 68.7% and from 72.4% to 82.9%, respectively, with one memory retained throughout.

**Final-round planning outcomes.** Table 8 compares the fifth round of SE-JEPA with the archived Frozen and AdaJEPA baselines. SE-JEPA reaches 75.1% (GD) and 75.8% (CEM), with higher success in both families and fewer replans overall. Relative to AdaJEPA, the observed overall gaps are 10.0 and 10.4 percentage points, while all-episode replans decrease by 18.8% and 17.6%. These gains combine a higher fraction of solved goals with a lower interaction cost. Table 9 reports every domain: SE-JEPA exceeds AdaJEPA in 12 of 14 domains with GD, and in 13 with CEM, tying on noise.

The baseline runs use the official shape-shift checkpoint released with AdaJEPA for both domain families and reset each episode. SE-JEPA uses our TLZ+ epoch-3 checkpoint and retains learned memory across episodes. Initial/goal pairs are matched, but the initial checkpoints and use of earlier experience differ, so these comparisons evaluate the complete systems. The within-system learning curves establish continued improvement in a mixed stream. A controlled comparison would be needed to determine whether learning in one task family directly improves the other.

### D.4. Planning efficiency

Figure 5 plots cumulative held-out success against the number of MPC replans. SE-JEPA pulls ahead within the first four replans and keeps the lead for the rest of the budget, showing how the advantage develops before the full interaction budget of 30 MPC replans is exhausted.

Table 7. Held-out visual success (%) per condition on the paired targets. All averages seven conditions; Shift excludes Original.

| Planner | Method | Original | Blur | Noise | Dark | Red agent | Red object | Red goal | All | Shift |
|---|---|---|---|---|---|---|---|---|---|---|
| CEM | Frozen | 66.0 | 56.0 | 62.7 | 70.7 | 50.0 | 14.0 | 36.7 | 50.9 | 48.3 |
| CEM | AdaJEPA | 79.3 | 78.0 | 76.7 | 80.0 | 71.3 | 14.0 | 41.3 | 63.0 | 60.2 |
| CEM | SE-JEPA | 82.7 | 86.4 | 86.7 | 88.0 | 79.6 | 47.1 | 68.9 | 77.0 | 76.1 |
| GD | Frozen | 59.3 | 48.7 | 55.3 | 58.7 | 50.7 | 6.7 | 29.3 | 44.1 | 41.6 |
| GD | AdaJEPA | 81.3 | 77.3 | 73.3 | 73.3 | 58.0 | 10.7 | 36.7 | 58.7 | 54.9 |
| GD | SE-JEPA | 88.0 | 88.7 | 84.9 | 87.3 | 74.7 | 44.4 | 61.8 | 75.7 | 73.6 |

Table 8. **Mixed 14-domain online results.** Success (%) and all-episode MPC replans, mean ± sample SD across three chains (SE-JEPA, round five) or three seeds (baselines). Each row has 2,100 episodes, split equally between the two families. Failed episodes count as 30 replans. The checkpoint and learning protocols are described above.

| Planner | Method | All success ↑ | Shape success ↑ | Visual success ↑ | Replans ↓ |
|---|---|---|---|---|---|
| GD | Frozen | $37.7 \pm 0.0$ | $42.0 \pm 0.0$ | $33.4 \pm 0.0$ | $21.29 \pm 0.00$ |
| GD | AdaJEPA | $65.1 \pm 0.7$ | $72.0 \pm 0.9$ | $58.2 \pm 0.7$ | $16.04 \pm 0.07$ |
| GD | **SE-JEPA** | $\mathbf{75.1 \pm 1.4}$ | $\mathbf{81.1 \pm 2.3}$ | $\mathbf{69.1 \pm 1.6}$ | $\mathbf{13.03 \pm 0.32}$ |
| CEM | Frozen | $38.6 \pm 0.9$ | $44.0 \pm 1.0$ | $33.2 \pm 2.5$ | $21.93 \pm 0.33$ |
| CEM | AdaJEPA | $65.4 \pm 0.7$ | $71.8 \pm 0.4$ | $59.0 \pm 1.4$ | $17.28 \pm 0.25$ |
| CEM | **SE-JEPA** | $\mathbf{75.8 \pm 0.8}$ | $\mathbf{82.9 \pm 1.0}$ | $\mathbf{68.7 \pm 1.9}$ | $\mathbf{14.24 \pm 0.18}$ |

### D.5. Deformable manipulation

Figure 6 shows the four-pass learning curves in the main paper. Table 10 adds standard errors to the deformable results of Table 1. Table 11 follows consolidation pass by pass at two slow learning rates. With $10^{-4}$, CD decreases at every pass on both tasks. Rope improves faster in the early passes at $2 \times 10^{-4}$; the two learning rates yield similar final goal errors after four passes on both deformable tasks.

### D.6. Component protocols and learning rates

All new component evaluations use GD. Within each task, variants share the initial checkpoint and the five-round learning protocol. Each evaluated memory is the final state of its own learning chain. Testing uses seeds 2000, 2100, and 2200, with 50 episodes per seed and domain: 1,050 evaluations for each method and chain. PushT includes the original appearance and six shifts, so these seven-condition means differ from the Shift column in Table 1. Replans average over all episodes, counting failures as 30.

The episodic-reset ablation starts each goal from the same checkpoint as the corresponding full model; it is distinct from the official-checkpoint benchmark. The shared-replay variant trains one slow predictor on stored trajectories, while EMA averages fast parameters with mixing coefficient $\alpha = 10^{-4}$. This coefficient is not a gradient learning rate. Removing maturity screening changes eligibility for retrieval but retains error-based predictor selection. The familiarity variants separately remove the read restriction, the direct-write restriction, or both, while keeping the same eligibility test for slow-predictor maturity.

Tables 12 and 13 report each available learning chain separately for each task. Most visual variants have two learned states; the full model, direct inheritance, doubled learning rate, and removal of both familiarity protections have three. The full model outperforms inheritance in every chain on both tasks. On PushT, the three-chain mean is 75.68% for the default and 75.87% for the doubled learning rate, while removal of both familiarity protections reduces it to 72.32%. Mean MPC replans are 13.21, 13.09, and 13.88, respectively, with failures included.

The retained episode records reproduce all 51 unique GD evaluation conditions in the supplied archive. Available raw visual initial and goal states agree across methods for each seed and condition. Historical full-model PushObj records retain episode outcomes but not complete state tensors, so their physical-state pairing cannot be independently rechecked from those records alone. Learning-curve measurements are online outcomes and are kept separate from these held-out evaluations.

For GD, disabling only familiar-read protection produces the same held-out outcomes as episodic reset across all 21 PushObj test batches. The GD visual results in Table 12 show that familiarity and maturity screening have distinct effects: the former protects familiar episodes, while the latter controls when learned predictors can compete for retrieval. The CEM familiarity results remain in Table 14; a second full-model CEM run on the same learning stream reached 80.19%, compared with 78.57% in the original run. Doubling the visual slow learning rate changes three-state

Table 9. **Mixed-stream success (%) for every domain.** Each entry averages 150 episodes under the protocol of Table 8. Original appearance and the T-shape block use different target sets and are retained as separate domains.

| | | GD | | | CEM | | |
|---|---|---|---|---|---|---|---|
| Family | Domain | Frozen | AdaJEPA | SE-JEPA | Frozen | AdaJEPA | SE-JEPA |
| Visual | Original | 50.0 | 70.7 | **78.7** | 46.7 | 72.0 | **78.0** |
| Visual | Blur | 32.0 | 72.7 | **78.0** | 30.7 | 70.0 | **72.7** |
| Visual | Noise | 44.0 | 76.7 | **78.7** | 41.3 | **73.3** | **73.3** |
| Visual | Dark | 48.0 | 72.0 | **76.0** | 48.7 | 73.3 | **74.7** |
| Visual | Red agent | 36.0 | 56.0 | **62.0** | 38.0 | 56.7 | **71.3** |
| Visual | Red object | 6.0 | 13.3 | **52.0** | 6.0 | 21.3 | **50.7** |
| Visual | Red goal | 18.0 | 46.0 | **58.7** | 21.3 | 46.0 | **60.0** |
| Shape | I | 40.0 | 56.7 | **82.7** | 26.0 | 54.0 | **88.0** |
| Shape | L | 52.0 | **83.3** | 82.0 | 49.3 | 82.7 | **90.0** |
| Shape | Square | 14.0 | 41.3 | **60.0** | 26.7 | 38.7 | **59.3** |
| Shape | Z | 34.0 | 78.7 | **79.3** | 42.7 | 78.7 | **83.3** |
| Shape | Small tee | 52.0 | **78.7** | 76.0 | 56.7 | 80.0 | **81.3** |
| Shape | Plus | 54.0 | 84.7 | **94.0** | 46.7 | 81.3 | **86.7** |
| Shape | T | 48.0 | 80.7 | **94.0** | 60.0 | 87.3 | **91.3** |

Table 10. Deformable manipulation: mean CD $\pm$ standard error over 50 instances. SE-JEPA uses its slow predictor after four passes through the same 50 instances.

| Method | Rope CD ↓ | Granular CD ↓ |
|---|---|---|
| DINO-WM | $0.491 \pm 0.059$ | $0.301 \pm 0.020$ |
| LeWM | $0.786 \pm 0.112$ | $0.322 \pm 0.018$ |
| DDP-WM | $0.677 \pm 0.086$ | $0.259 \pm 0.012$ |
| AdaJEPA | $0.629 \pm 0.096$ | $0.304 \pm 0.019$ |
| SE-JEPA | $\mathbf{0.252 \pm 0.045}$ | $\mathbf{0.220 \pm 0.012}$ |

Table 11. CD of SE-JEPA after each consolidation pass at two slow learning rates (mean $\pm$ standard error, 50 instances).

| | lr $10^{-4}$ | | lr $2 \times 10^{-4}$ | |
|---|---|---|---|---|
| Pass | Rope | Granular | Rope | Granular |
| 1 | $0.469 \pm 0.100$ | $0.257 \pm 0.014$ | $0.416 \pm 0.070$ | $0.260 \pm 0.012$ |
| 2 | $0.342 \pm 0.054$ | $0.235 \pm 0.012$ | $0.286 \pm 0.050$ | $0.241 \pm 0.012$ |
| 3 | $0.282 \pm 0.033$ | $0.224 \pm 0.010$ | $0.283 \pm 0.037$ | $0.247 \pm 0.012$ |
| 4 | $0.252 \pm 0.045$ | $0.220 \pm 0.012$ | $0.243 \pm 0.033$ | $0.243 \pm 0.013$ |

mean success by $-1.05$ points with CEM.

## D.7. Transferring fast weights across goals

We tested whether the parameter update from one episode helps on the next. Let $s_e$ be the offset from the pretrained weights at the start of episode $e$, and $\Delta_e$ the fast update in that episode. We set $s_{e+1} = s_e + w\Delta_e$, where $w = 0$ resets to the pretrained model and $w = 1$ retains the full update. Each variant sees the same 100 PushObj goals in five blocks of 20 (I, square, T, I, square). Keeping the full update loses more goals than it gains relative to reset (Table 2, two-sided exact McNemar $p = 0.029$). Smaller weights give success rates closer to those of independent episodic reset.

## D.8. A seven-round shape stream

A second PushObj stream runs seven rounds with a slow learning rate of $2 \times 10^{-4}$, one chain per planner. Each round has 50 episodes per shape, drawn without replacement and excluding the held-out sources, for 2,450 episodes per planner. Online success rises from 70.6% in the first round to 78.6% in the seventh with GD, and from 72.3% to 78.3% with the sampling-based CEM planner.

The episode logs record the fast-update fingerprint of every episode. Figure 9 shows the cosine between consecutive fingerprints: the mean is 0.018 for both planners. Absolute cosine is below 0.1 for 96.3% of GD pairs and 96.8% of CEM pairs. These are descriptive statistics from one sequential stream per planner. These measurements support consolidating transition data rather than accumulating episode-specific parameter updates (Sec. 3.1).

## D.9. Memory use across episodes

SE-JEPA learns selectively. In a seven-round PushObj stream of 2,450 episodes per planner, it writes only 25% (GD) to 27% (CEM) of episodes to memory; the rest are familiar to the pretrained model, still pending, or rejected by validation. About one quarter of episodes retrieve a slow predictor; the others adapt from the pretrained model.

Selective writing leaves familiar dynamics to the pretrained model, whose original weights remain fixed. On the four pretraining shapes, SE-JEPA reaches 86.0% held-out success with CEM and 88.4% with GD, against 82.3% and 84.8% for AdaJEPA. Separate slow predictors learn recurring changes from replayed trajectories.

Table 12. **Complete GD component results.** Each entry is success (%) / mean MPC replans over 1,050 held-out evaluations. Each column preserves its own fifth-round endpoint. C1–C3 are learning chains; dashes denote unrun configurations. The single PushT episodic-reset evaluation is reused across columns and is not counted as three independent runs.

| | PushObj | PushT | | |
|---|---|---|---|---|
| Variant | C1 | C1 | C2 | C3 |
| Full SE-JEPA | 81.43 / 11.37 | 74.38 / 13.54 | 76.57 / 12.93 | 76.10 / 13.17 |
| Episodic reset | 76.95 / 12.54 | 65.90 / 15.47 | 65.90 / 15.47 | 65.90 / 15.47 |
| Inherit fast weights | 64.67 / 16.12 | 29.14 / 24.82 | 40.19 / 22.33 | 25.81 / 25.38 |
| Shared predictor: EMA | 75.71 / 12.84 | 66.10 / 15.42 | 66.29 / 15.30 | — |
| Shared predictor: replay | 81.24 / 11.41 | 76.19 / 13.11 | 75.14 / 13.26 | — |
| No maturity screening | 80.86 / 11.46 | 76.95 / 12.90 | 78.10 / 12.63 | — |
| No familiar-read protection | 76.95 / 12.54 | 74.67 / 13.43 | 76.19 / 13.16 | — |
| No familiar-write protection | 79.43 / 11.76 | 71.62 / 13.98 | 73.81 / 13.66 | — |
| Neither familiarity protection | 83.14 / 10.90 | 72.76 / 13.87 | 73.14 / 13.47 | 71.05 / 14.30 |
| Full SE-JEPA: fixed slow learning rate | | | | |
| $5 \times 10^{-5}$ | 79.62 / 11.79 | 75.24 / 13.36 | 73.05 / 13.65 | — |
| $10^{-4}$ (default) | 81.43 / 11.37 | 74.38 / 13.54 | 76.57 / 12.93 | 76.10 / 13.17 |
| $2 \times 10^{-4}$ | 82.38 / 11.26 | 74.86 / 13.35 | 76.86 / 12.86 | 75.90 / 13.07 |
| $3 \times 10^{-4}$ | 80.57 / 11.72 | 74.29 / 13.66 | 78.10 / 12.78 | — |
| $5 \times 10^{-4}$ | 78.38 / 12.40 | 71.62 / 14.29 | 74.38 / 13.53 | — |

Table 13. **Three-chain check of direct weight inheritance.** Success is measured on 1,050 held-out evaluations per chain. The mean combines 3,150 evaluations of repeated test targets, not 3,150 distinct physical scenes. Replans include failures.

| Task | Method | C1 | C2 | C3 | Mean success ↑ | Mean replans ↓ |
|---|---|---|---|---|---|---|
| PushObj | Full SE-JEPA | 81.43 | 81.14 | 82.19 | 81.59 | 11.30 |
| PushObj | Inherit fast weights | 64.67 | 67.05 | 66.10 | 65.94 | 15.84 |
| PushT | Full SE-JEPA | 74.38 | 76.57 | 76.10 | 75.68 | 13.21 |
| PushT | Inherit fast weights | 29.14 | 40.19 | 25.81 | 31.71 | 24.18 |

Table 14. Effect of familiarity protection on PushObj success (%). Each run uses the same five-round learning stream and 1,050 held-out evaluations across seven shapes. Dashes indicate configurations with no reported test.

| Familiarity protection | GD | CEM |
|---|---|---|
| Full SE-JEPA | 81.43 | 78.57 |
| Disable read protection only | 76.95 | — |
| Disable write protection only | 79.43 | — |
| Disable both protections | 83.14 | 79.62 |

# E. Qualitative Examples and Reproducibility

## E.1. Rigid-object planning sequences

Figure 3 selects two GD episodes where Frozen, DINO-WM, and AdaJEPA fail and SE-JEPA succeeds. The unseen-I example uses the paired shape evaluation. Its DINO-WM frame replays recorded actions; the same procedure exactly reproduces all 31 archived Frozen frames and states.

The red-object example comes from round five of the 14-domain stream (SE-JEPA chain two, sample 30), matched exactly in both initial and goal state to DINO-WM sample 31. Frozen and AdaJEPA use their seed-100 runs and the official shape-shift checkpoint; SE-JEPA uses the mixed-stream checkpoint and accumulated memory described in Appendix D. The DINO-WM final frame comes from a new frozen-model run with 30 MPC replans, preserving the original epoch-three checkpoint, target and seed mapping. The other three final frames replay their original actions. This is a comparison of complete systems whose initial checkpoints and learning histories differ.

Figure 3 labels MPC replans. All baseline runs fail after 30 replans of five environment actions, exhausting the same 150-action budget. SE-JEPA succeeds after six replans in each row. The examples were selected for these outcomes; aggregate success uses the complete evaluation sets.

Figures 10 and 11 show the square and red-agent CEM examples from the original execution archive. Both use the first held-out target of test seed 2000. We retain the full frame strips, termination markers, prediction-loss curves, and explanatory notes. The label Ours denotes SE-JEPA; Replay identifies frames reconstructed in the original simu-

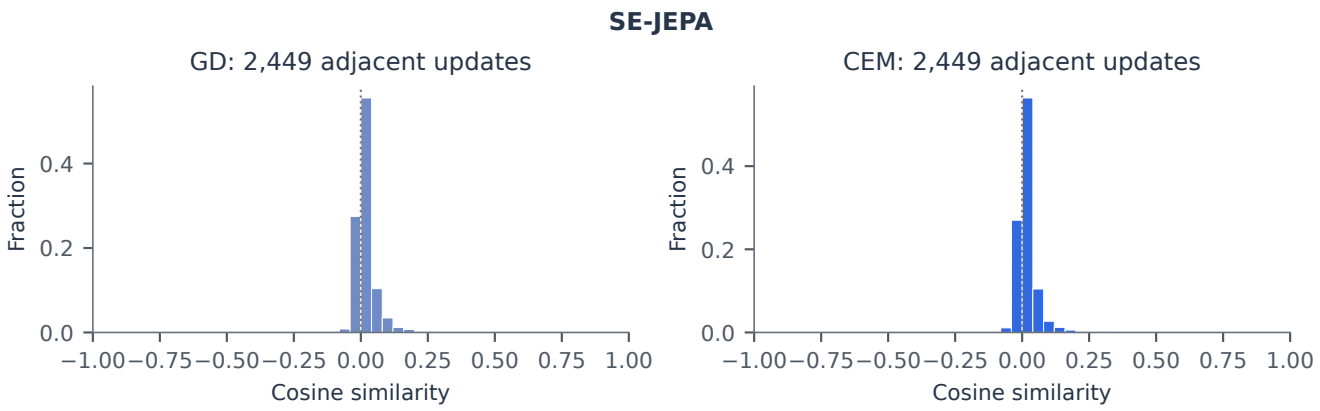



Figure 9. Cosine similarity between consecutive fast-update fingerprints in the seven-round shape stream (2,449 pairs per planner).

lator from the recorded actions. Gray frames hold the terminal state once the episode has reached its goal.

On the unseen square, SE-JEPA reaches the goal after five replans, compared with eight for AdaJEPA; Frozen exhausts all 30 replans without success. On the red-agent shift, SE-JEPA succeeds at replan ten while both baselines exhaust the budget. These cases show two benefits of memory: reaching a shared goal sooner and solving a goal that episode-local adaptation misses.

The plotted loss is prediction MSE over up to five recent transitions in the visual and proprioceptive latents, excluding action dimensions. Each curve is evaluated with its own method's weights, so absolute loss levels are not directly comparable across methods. The curves show how each method's prediction error changes during execution; task success is the common measure of planning performance. In particular, AdaJEPA reduces its prediction loss on the red-agent episode but still misses the target, whereas SE-JEPA achieves the desired configuration.

## E.2. Deformable rollout sequences

Figure 12 shows two complete deformable sequences: rope example 1 and granular example 2. Each row contains the start, every third push, and the goal within the same 15-push budget. These instances come from the repeated 50-instance stream and were selected where both baselines fail and the consolidated predictor succeeds. The examples use the third-pass slow predictor without within-episode adaptation; Table 1 reports fourth-pass aggregates. DINO-WM frames replay recorded actions in FleX; AdaJEPA and SE-JEPA frames were saved during planning. Final CD values and success markers are retained from the original logs.

The first rope example starts at CD 1.72. Both baselines finish at 1.54, while SE-JEPA reaches 0.31 and the target success threshold. In granular example 2, final CD is 0.33 for both baselines and 0.14 for SE-JEPA. The sequences show how replayed experience improves the geometry of the final configuration under a common action budget, even with fast adaptation disabled.

## E.3. Reproducibility

Source manifests link archived episode records, simulator images, outcomes, checksums, and crops. Appendix D specifies mixed-stream protocols and checkpoint provenance. The release will include code, configurations, learned states, and the supporting evaluation records.

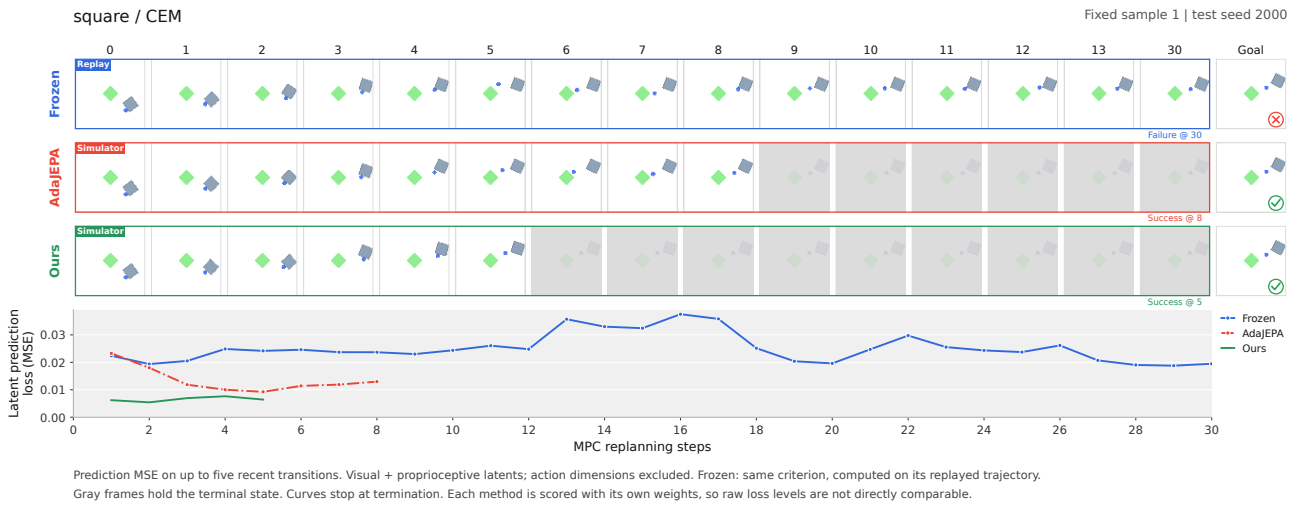



Figure 10. **Unseen-square planning under CEM.** The original trajectory strip includes replans 0–13 and 30, followed by the goal. SE-JEPA succeeds at replan five and AdaJEPA at replan eight; Frozen fails within 30 replans. Loss traces stop at termination. The in-panel notes specify frame provenance and the prediction-loss definition.

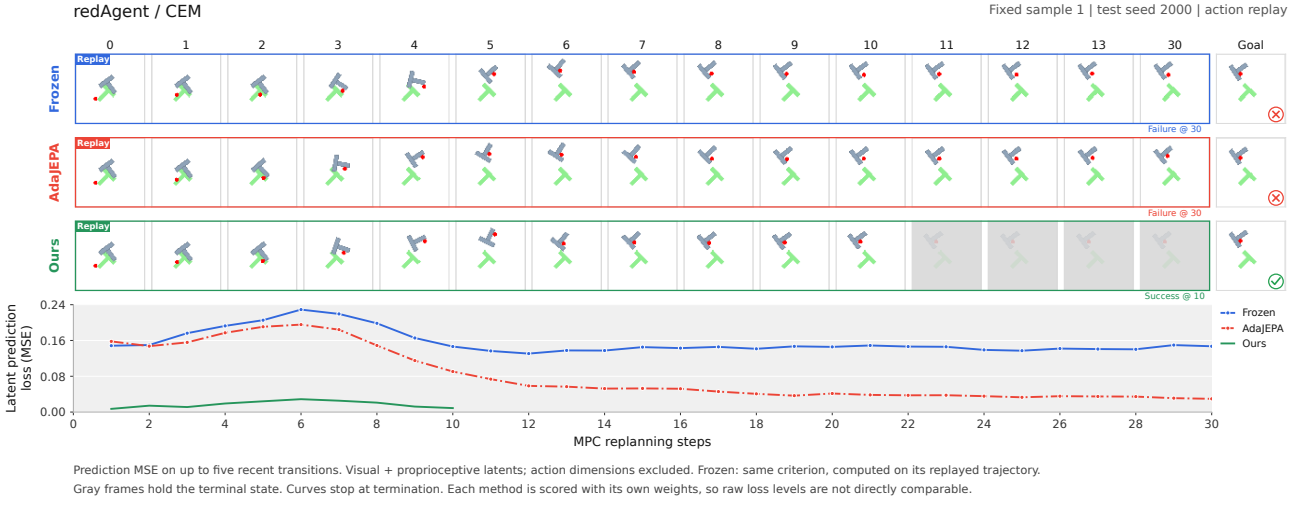



Figure 11. **Planning through a red-agent appearance shift under CEM.** SE-JEPA reaches the goal at replan ten; Frozen and AdaJEPA reach the 30-replan limit. The trajectory images are simulator replays of recorded actions. Falling prediction loss alone does not guarantee reaching the goal, as the AdaJEPA trajectory illustrates.

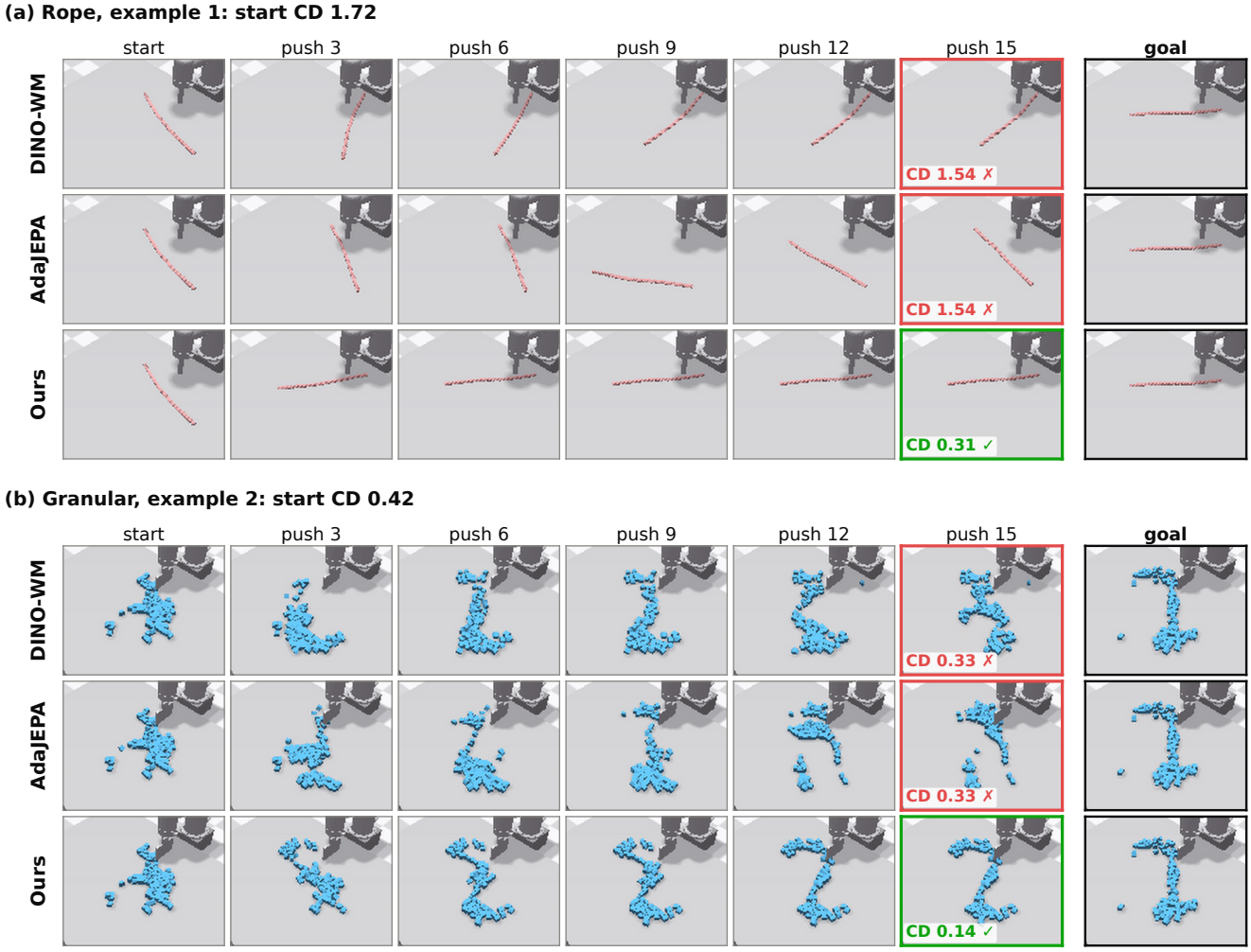



Figure 12. **Rope and granular execution sequences.** Top: rope example 1. Bottom: granular example 2. All methods share the initial configuration and goal. Columns show pushes 0, 3, 6, 9, 12 and 15, then the goal. Final-frame labels report CD (lower is better) and success. SE-JEPA reaches both goals with its consolidated slow predictor; both baselines fail.